

Blackthorn

Security Review For Morpho

Collaborative Audit Prepared For: **Morpho**

Lead Security Expert(s): **0x73696d616f**

hyh

xiaoming90

Date Audited: **July 30 - August 2, 2026**

Introduction

BluePublicAllocator lets anyone (not only the vault's allocators) reallocate a Vault V2's liquidity between Morpho Blue markets reached through the MorphoMarketV1AdapterV2, within the vault's caps, and the public allocator's own limits. Vault allocators configure which adapters are active, which markets can be deallocated from, each market's absolute cap, and whether idle assets can be allocated publicly. Public callers must pay the vault's penalty on each `reallocate` or `allocateFromIdle` call, a proportion of the assets moved, paid in the market's loan token and sent directly to the vault (a donation).

Scope

Repository: [morpho-org/vault-v2](https://github.com/morpho-org/vault-v2)

Audited Commit: `10f169717ae652e346d06f6ce1851578b905579e`

Final Commit: `2b139002feaf4631f1c63d880380ca2278df7aec`

Files:

- `src/periphery/blue-public-allocator/BluePublicAllocator.sol`
- `src/periphery/blue-public-allocator/interfaces/IBluePublicAllocator.sol`

Final Commit Hash

`2b139002feaf4631f1c63d880380ca2278df7aec`

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.

- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
1	4	7

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Security Experts Dedicated to This Review

@0x73696d6166

A handwritten signature in white ink on a black background. The signature is stylized, starting with a large 'S' that loops around, and the name 'Simao' is written in a cursive script.

@hyh

A handwritten signature in white ink on a black background. The signature is stylized, starting with a large 'h' that loops around, and the name 'hyh' is written in a cursive script.

@xiaoming90

A handwritten signature in white ink on a black background. The signature is stylized, starting with a large 'X' that loops around, and the name 'Xiaoming90' is written in a cursive script.

Issue H-1: Real exposure to a compromised oracle is allowed to surpass Vault level caps by magnitudes [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit/2026-07-morpho-vault-v2-update-july-30th/issues/7>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

Suppose there is a collateral listed in Morpho Blue and having attacker controlled oracle (e.g. hacked at some point after listing). A compromised oracle used to cost a Vault only what an allocator had put in the rogue market: no allocator will manually supply again after bad debt. BluePublicAllocator (BPA) removes that safety net, as anyone can refill.

Cap accounting compares live value to a stale recorded allocation, so supplying exactly the bad debt destroyed amount posts no change. With attacker controlled oracle bad debt can be created and removed in turns, so an attack scenario can be: borrow, drop price, liquidate the default, refill, raise price, repeat until everything BPA can reach is stolen, many cycles atomically in a single transaction. Both BPA and Vault level caps pass at every step, so the cap itself no longer bounds the cumulative exposure.

Vulnerability Detail

Affected oracle can continue to behave normally waiting for caps to be allocated for it / allocation to happen (depending on when the hack took place vs collateral lifecycle with Blue). When it happens, it wasn't possible to harm beyond the allocation, i.e. once cap is set the attacker can wait for the allocation within the cap, then manipulate the oracle and borrow maximally and that's it: they could then drop the price, create bad debt, and write it down, but no allocator would deliberately repeat allocation after that (unless colluded, but it's a less probable scenario), so there were little incentives for attacker to go this direction. Max damage was the allocation actually happened.

Now, with BPA, the attacker can repeat bad debt realization, allocation and borrowing

atomically until all the allocator configured paths are exhausted, i.e. the attacker can seize all the Vault funds exposed to this market allocation via current set of flags. It will cost the `nativePenalty` and gas (independent of assets, but it barely matters here) per turn, but allows for stealing all the exposed capital, so it's profitable.

Morpho Blue deducts bad debt from `totalSupplyAssets` when a liquidated borrower has no collateral left ([Morpho.sol#L391-L401](#)). Live value falls at once, but `caps[id].allocation` updates only when the adapter runs, as live value minus stale record ([MorphoMarketV1AdapterV2.sol#L194-L202](#)). If record is S, deduction is L, refill is L then `change = 0`. The Vault applies zero to all three ids ([VaultV2.sol#L588-L598](#)); the periphery check after the call reads the same number ([BluePublicAllocator.sol#L138](#)). `mintedShares >= assets` does not block the loop: the deduction cuts assets while shares stay, so new supplies mint at a depressed price ([MorphoMarketV1AdapterV2.sol#L190](#)).

The cycle: `allocateFromIdle(S)` -> worthless collateral at an inflated price -> borrow S -> crash the price -> liquidate your own position. Size each refill as `cap - expectedSupplyAssets()` so the record lands back on the cap. Only cycle 1 must fit the absolute and relative caps; a relative cap limits cycle size, never the total. Cost: gas plus the flat fee. Capital comes from idle and, through `reallocate()`, can come from any market open as a source. Each deduction collapses the share price, but the loop ends on Vault liquidity and gas long before `totalSupplyShares` nears `uint128`.

The attack runs atomically and the Vault never sees the loss: `firstTotalAssets` freezes the relative cap denominator and `_totalAssets` for the rest of the transaction ([VaultV2.sol#L652](#), [VaultV2.sol#L665](#)). At relative cap WAD the loop also works across transactions, with only atomicity defeats relative caps below WAD.

Preconditions: the market is listed with Vault caps, an allocator set `isActiveAdapter`, can `AllocateFromIdle` and a BPA's `absoluteCap`. The adapter constrains only `loanToken` and `irm`, the oracle is unconstrained, has to be attacker controlled, can change behavior after listing.

Impact

Before, an oracle compromise cost at most the Vault cap bounded allocation. Now, it costs everything BPA can reach, for gas plus a flat fee, atomically: no depositor exit, sentinel cap decrease or allocator rebalance can intervene the attack.

i.e. "the amount of Vault funds exposed to public allocator via current settings" factually

replaces Vault's collateral cap, as bad debt realization allows avoiding hitting it, and repeating the allocation continuously.

Code Snippet

Adapter returned delta absorbs the loss, live value minus stale record:

MorphoMarketV1AdapterV2.sol#L194-L202

```
uint256 oldAllocation = allocation(marketParams);
uint256 newAllocation = expectedSupplyAssets(marketId);
updateList(marketId, oldAllocation, newAllocation);
return (ids(marketParams), int256(newAllocation) - int256(oldAllocation));
```

Proof of Concept

test/periphery/RogueMarketRepeatDrainPoCTest.sol, run with FOUNDRY_PROFILE=no_via_i
r forge test --match-contract RogueMarketRepeatDrainPoCTest. Caps at 1e18, 3e18 idle:
three cycles steal 3x the cap with the record at the cap after each refill, the baseline
reverts on AbsoluteCapExceeded without the deduction. The market state left by a past
deduction is written with vm.store, all attack calls are ordinary.

```
// SPDX-License-Identifier: GPL-2.0-or-later
pragma solidity ^0.8.28;

import "../integration/MorphoMarketV1IntegrationTest.sol";
import {BluePublicAllocator} from
↳ "../src/periphery/blue-public-allocator/BluePublicAllocator.sol";
import {ErrorsLib} from "../src/libraries/ErrorsLib.sol";

/// @dev RogueMarketRepeatDrain: public refill after write-off pins recorded
↳ allocation at CAP;
/// three cycles steal ~3x CAP. Morpho position=2, market=3; adapter marketIds=4,
↳ supplyShares=5; VaultV2 caps=18.
contract RogueMarketRepeatDrainPoCTest is MorphoMarketV1IntegrationTest {
    using MarketParamsLib for MarketParams;

    BluePublicAllocator internal bpa;
```

```

address internal attacker = makeAddr("attacker");

uint256 internal constant CAP = 1e18;
uint256 internal constant DEPOSIT = 3e18;
uint256 internal constant RESIDUAL = 1e15;

function setUp() public override {
    super.setUp();
    bpa = new BluePublicAllocator();

    vm.prank(curator);
    vault.submit(abi.encodeCall(IVaultV2.setIsAllocator, (address(bpa), true)));
    vault.setIsAllocator(address(bpa), true);

    vm.prank(curator);
    vault.decreaseAbsoluteCap(expectedIdData2[2], CAP);

    vm.startPrank(allocator);
    bpa.setIsActiveAdapter(address(vault), address(adapter), true);
    bpa.setCanAllocateFromIdle(address(vault), true);
    bpa.setAbsoluteCap(address(vault), address(adapter), marketParams2, CAP);
    vm.stopPrank();

    vault.deposit(DEPOSIT, address(this));
    underlyingToken.approve(address(morpho), type(uint256).max);
    _seedLegacyPosition();
}

/// @dev Past CAP allocation then write-off: recorded CAP-RESIDUAL, dust
    ↪ assets, 1000x shares.
function _seedLegacyPosition() internal {
    uint256 shares = RESIDUAL * 1000;
    Id id = marketParams2.id();
    bytes32 marketSlot = keccak256(abi.encode(id, 3));
    vm.store(address(morpho), marketSlot, bytes32((shares << 128) | RESIDUAL));
    vm.store(address(morpho), bytes32(uint256(marketSlot) + 2),
        ↪ bytes32(block.timestamp));
}

```



```

vm.store(address(morpho), keccak256(abi.encode(address(adapter),
    ↪ keccak256(abi.encode(id, 2)))), bytes32(shares)));
vm.store(address(adapter), keccak256(abi.encode(Id.unwrap(id), 5)),
    ↪ bytes32(shares)));
vm.store(address(adapter), bytes32(uint256(4)), bytes32(uint256(1)));
vm.store(address(adapter), keccak256(abi.encode(uint256(4)),
    ↪ Id.unwrap(id)));
vm.store(address(vault), keccak256(abi.encode(expectedIds2[2], 18)),
    ↪ bytes32(CAP - RESIDUAL));

assertEq(vault.allocation(expectedIds2[2]), CAP - RESIDUAL, "seed:
    ↪ recorded");
assertApproxEqAbs(adapter.expectedSupplyAssets(Id.unwrap(id)), RESIDUAL,
    ↪ 1e6, "seed: live");
}

function test_PoC_baseline_capsBindWithoutWriteOff() public {
    Id id = marketParams2.id();
    uint256 refill = CAP - adapter.expectedSupplyAssets(Id.unwrap(id)) - 100;
    bpa.allocateFromIdle(address(vault), address(adapter), marketParams2,
        ↪ uint128(refill));
    assertApproxEqAbs(vault.allocation(expectedIds2[2]), CAP, 1e9, "recorded at
        ↪ cap");

    vm.expectRevert(ErrorsLib.AbsoluteCapExceeded.selector);
    bpa.allocateFromIdle(address(vault), address(adapter), marketParams2,
        ↪ uint128(1e12));
}

function test_PoC_rogueMarketRepeatDrain() public {
    Id id = marketParams2.id();

    for (uint256 i; i < 3; i++) {
        uint256 refill = CAP - adapter.expectedSupplyAssets(Id.unwrap(id)) -
            ↪ 100;
        bpa.allocateFromIdle(address(vault), address(adapter), marketParams2,
            ↪ uint128(refill));
    }
}

```

```

    oracle.setPrice(ORACLE_PRICE_SCALE);
    uint256 collateral = 2 * refill;
    deal(address(collateralToken), attacker, collateral);
    vm.startPrank(attacker);
    collateralToken.approve(address(morpho), collateral);
    morpho.supplyCollateral(marketParams2, collateral, attacker, hex "");
    morpho.borrow(marketParams2,
        ↪ underlyingToken.balanceOf(address(morpho)), 0, attacker, attacker);
    underlyingToken.approve(address(morpho), 10);
    oracle.setPrice(1);
    morpho.liquidate(marketParams2, attacker, collateral, 0, hex "");
    vm.stopPrank();

    assertApproxEqAbs(vault.allocation(expectedIds2[2]), CAP, 1e9,
        ↪ "recorded pinned at cap");
}

assertApproxEqAbs(underlyingToken.balanceOf(attacker), DEPOSIT, 4 *
    ↪ RESIDUAL, "attacker holds ~3x cap");
assertLe(underlyingToken.balanceOf(address(vault)), 4 * RESIDUAL, "idle
    ↪ drained");
vm.expectRevert();
vault.withdraw(1e18, address(this), address(this));
}
}

```

Tool Used

Manual Review, Foundry

Recommendation

Consider capping committed principal instead of current value, with tracking cumulative public inflow per id against absoluteCap, e.g.:

[BluePublicAllocator.sol#L137-L138](#)

```

- bytes32 allocateId = vaultBlueId(adapter, marketParams);
- require(IVaultV2(vault).allocation(allocateId) <=
↪ absoluteCap[vault][allocateId], AbsoluteCapExceeded());
+ bytes32 allocateId = vaultBlueId(adapter, marketParams);
+ publicPrincipal[vault][allocateId] += assets;
+ require(publicPrincipal[vault][allocateId] <= absoluteCap[vault][allocateId],
↪ AbsoluteCapExceeded());

```

The counter shouldn't decrease on public outflow: if it did, an attacker with `canDeallocate` on the rogue market and headroom in a buffer market could shuffle the same amount to recycle the allowance and keep draining. Only an allocator resets the counter (deliberate, so re enabling public inflow is manual and conscious).

A consumable inflow budget refilled only by an allocator have the similar effect, bounding cycle count.

Discussion

MathisGD

what is the fundamental difference between this issue #5? it highlights the behavior of the allocation cap w.r.t the loss?

Cap accounting compares live value to a stale recorded allocation, so supplying exactly the bad debt destroyed amount posts no change. With attacker controlled oracle bad debt can be created and removed in turns, so an attack scenario can be: borrow, drop price, liquidate the default, refill, raise price, repeat until everything BPA can reach is stolen, many cycles atomically in a single transaction. Both BPA and Vault level caps pass at every step, so the cap itself no longer bounds the cumulative exposure.

just want to be very clear on that there is nothing specific to the BPA here, allocators (which act without timelock) could already do all this

dmitriia

The difference is the attacks: in #5 a liquidator who also have an affiliated position in a

market with bad debt, can shift the loss by allocating the Vault idle there just before bad debt realization. One time, no caps breached. This one is repeated atomic drain based on bad debt not moving allocation (not needed in #5), with breaching the caps (basically it doesn't matter what cumulative cap Vault have for the affected oracle's markets, even smallish cap can be reused many times up to block gas constraints) by arbitrary magnitude, depending only on BPA settings.

Both can be done before with a colluded allocator, which is a quite strong prerequisite. BPA introduction removed this precondition in both cases, now some BPA allowances can be used permissionlessly (which are natural to have, i.e. one can't say "but there will be no allowances", as there will be no BPA use case without some allowances, i.e. some paths will be open to the public and more will naturally be opened over time). And the thing is it's not really possible to say where is the fine line allowance wise. This issue is the most dangerous because of the fact the cap size doesn't matter anymore, opening just a little bit can be enough.

peyha

We acknowledge the issue. While theoretically feasible, this attack looks very unlikely because it requires oracle manipulation which is already a huge assumption (it can already drain of the current vault allocation within the market without any public allocator). Allocators could also react by setting the vault market cap or the public allocator market cap to 0.

dmitriia

Agree that oracle manipulation is very rare and significant precondition and it allows drain now up to caps. This one shows that caps can be extended to BPA reachable paths, which adds a dimension to risk management of a Vault, and the corresponding attack can be executed in one transaction, so allocators might not have any room to react, similarly to the current case.

Issue M-1: Flat native penalty allows for profitable Blue interest rate manipulations and griefing [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-07-morpho-vault-v2-update-july-30th/issues/3>

Summary

Both public allocation functions charge a fixed fee independent of `assets`: moving one `wei` and moving the whole balance cost the same, and the same headroom can be bought again every block. When the fee is low against the benefits of the move, the caller pays gas and the flat fee only profiting off vault depositors.

Vulnerability Detail

First, a griefing, large reallocation whose cost to the vault exceeds the fee: the attacker pays gas and the fee, depositors absorb the impact.

Second, every market the allocator can reach use adaptive rate curve (`irm == adaptiveCurveIrm`): a borrower pushes vault capital into the market they borrow from, lowers utilization and their rate, and repeats after any reverse. E.g., given a reaction window of one hour, for any fixed fee of `0.1 ETH` there is an `assets` at which the move pays: profit scales with size, cost does not.

Impact

Flat 'nativePenalty' is paid exactly when it is cheap and allows for abuse at scale. Manipulative reallocations stay repeatable after every manual allocator correction.

Code Snippet

Fixed price, no `assets` term: [BluePublicAllocator.sol#L106-L108](#)

```
require(msg.value == vaultData[vault].nativePenalty, IncorrectNativePenalty());
if (msg.value > 0) vaultData[vault].accruedNativePenalty += uint120(msg.value);
```

Tool Used

Manual Review

Recommendation

Consider charging proportionally to the amount moved, e.g.:

[BluePublicAllocator.sol#L106](#)

```
- require(msg.value == vaultData[vault].nativePenalty, IncorrectNativePenalty());
+ uint256 penalty = vaultData[vault].nativePenalty + uint256(assets) *
  ↪ vaultData[vault].penaltyPerAsset / WAD;
+ require(penalty <= type(uint120).max && msg.value == penalty,
  ↪ IncorrectNativePenalty());
```

Discussion

MathisGD

at the end we changed the penalty mechanism to be relative (and paid in underlying token), so it partially fixes this (and we can't do much better simply)

<https://github.com/morpho-org/vault-v2/pull/966>

dmitriia

<https://github.com/morpho-org/vault-v2/pull/966> looks ok

Issue M-2: A liquidator can shift their Morpho Blue bad debt loss to the Vault [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit/2026-07-morpho-vault-v2-update-july-30th/issues/5>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

Morpho Blue spreads bad debt pro rata over supply on liquidation. A caller who is about to liquidate bad debt and who is affiliated to a supplier locked in the corresponding market can first push the Vault's idle into it with `allocateFromIdle()`, then withdraw their supply at par with the liquidity they created, then liquidate the underwater borrower, all atomically. Vault and the remaining depositors will bear the bad debt loss.

Vulnerability Detail

When bad debt liquidator has an affiliated position in the corresponding Blue market, which is being utilized highly enough, so they cannot withdraw fully, before they had no options other than withdrawing all they can and bearing the loss with the remainder (there is generally no point in postponing the liquidation as it is free for all and profitable), and now they can run `allocateFromIdle()` to that market and withdraw own funds up to fully.

As bad debt socialization is pro rata over current supply they could shift losses partially to stale depositors of the market before, and now they can shift losses to them and also all the depositors of the Vault proportionally to idle funds size.

E.g., suppose market supplied 10m1n and fully borrowed, underwater debt 1m1n, caller holds 2m1n, Vault idle 5m1n: doing nothing costs the caller 200k, and without `BluePublicAllocator` all they could do is to bear this loss as no withdrawal was possible and liquidation is public. Liquidator can now atomically do instead: `allocateFromIdle()` of 5m1n, withdraw 2m1n, liquidate. The Vault absorbs about 385k loss. It's the case where

fixed nativePenalty is paid exactly when it is cheap vs the benefits, with Vault depositors paying for liquidator's additional gain, $200k - \text{nativePenalty} - \text{gas costs}$.

Impact

Vault's depositors absorb a loss that would have fallen on the liquidator, proportionally to the Vault's idle funds moved in. The prerequisites are liquidator having an affiliated existing position and an underwater borrower being liquidated.

Code Snippet

On liquidation bad debt deduces assets against unchanged shares:

[Morpho.sol#L391-L401](#)

```
if (position[id][borrower].collateral == 0) {
    badDebtShares = position[id][borrower].borrowShares;
    badDebtAssets = UtilsLib.min(
        market[id].totalBorrowAssets,
        badDebtShares.toAssetsUp(market[id].totalBorrowAssets,
            ↪ market[id].totalBorrowShares)
    );
    market[id].totalBorrowAssets -= badDebtAssets.toUint128();
    market[id].totalSupplyAssets -= badDebtAssets.toUint128();
}
```

The liquidator can atomically enter the Blue market above with Vault's idle funds just before bad debt realization: [BluePublicAllocator.sol#L135-L138](#)

```
IVaultV2(vault).allocate(adapter, abi.encode(marketParams), assets);

bytes32 allocateId = vaultBlueId(adapter, marketParams);
require(IVaultV2(vault).allocation(allocateId) <= absoluteCap[vault][allocateId],
    ↪ AbsoluteCapExceeded());
```

Tool Used

Manual Review

Recommendation

Consider bounding public inflow with a remaining budget per market per block: store `limit`, `remaining`, and the last block; the same block only decreases `remaining`, new block advances the saved block, sets `remaining` to `limit`, then decreases, with zero `limit` closing public inflow, e.g.:

```
error InflowBudgetExceeded();

struct FlowBudget {
    uint128 limit;      // set by allocator; zero closes public inflow
    uint128 remaining;  // residual for the current block
    uint32 block_;      // last block that consumed
}

mapping(address vault => mapping(bytes32 id => FlowBudget)) internal inflowBudget;

function setInflowBudget(address vault, address adapter, MarketParams calldata
    ↪ marketParams, uint128 newLimit)
    external
{
    require(IVaultV2(vault).isAllocator(msg.sender), Unauthorized());
    FlowBudget storage b = inflowBudget[vault][vaultBlueId(adapter, marketParams)];
    b.limit = newLimit;
    b.remaining = newLimit;
    b.block_ = uint32(block.number);
    emit SetInflowBudget(msg.sender, vault, adapter, marketParams, newLimit);
}

function consumeInflowBudget(address vault, bytes32 id, uint128 assets) internal {
    FlowBudget storage b = inflowBudget[vault][id];
    uint32 bn = uint32(block.number);
    if (b.block_ != bn) {
        b.block_ = bn;
        b.remaining = b.limit; // new block: seed from limit
    }
    require(b.remaining >= assets, InflowBudgetExceeded());
    b.remaining -= assets;
}
```

```
}
```

Both entry points consume before the allocate leg: [BluePublicAllocator.sol#L135](#)

```
+ bytes32 allocateId = vaultBlueId(adapter, marketParams);  
+ consumeInflowBudget(vault, allocateId, assets);  
  IVaultV2(vault).allocate(adapter, abi.encode(marketParams), assets);
```

This way only a bounded share of the loss can be shifted per block. The liquidator cannot wait and slowly withdraw `limit` each block, as someone else would execute the liquidation, while honest reallocator can.

If the penalty stays in absolute terms, which isn't advised, it should be reduced correspondingly. Penalty proportional to the assets needs no additional adjustment.

Discussion

MathisGD

we acknowledge this issue. we consider that it's a clear tradeoff when using the PA for curators/allocators. the speed limit recommendation is interesting but would also make using/integrating the contract much more complex

Issue M-3: BluePublicAllocator settings that raise risk take effect at once [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit/2026-07-morpho-vault-v2-update-july-30th/issues/8>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

VaultV2 timelocks changes that raise risk, so users can withdraw first.

BluePublicAllocator doesn't follow that: enabling an adapter, a source market, a maximum cap, idle allocation or a zero penalty is instant and open to allocators only. Depositors get no window, in the worst case an attacker acts in the same block right after the enabling transaction, and withdrawals can be instantly blocked.

Vulnerability Detail

VaultV2 timelocks every risk increase, including `setIsAllocator()` ([VaultV2.sol#L381-L385](#)) and `increaseAbsoluteCap()` ([VaultV2.sol#L526-L533](#)). The timelocked grant depositors observe is `setIsAllocator(bluePublicAllocator, true)`; at that moment every periphery map is still zero. The real grant happens later, at once, when any allocator writes the periphery settings ([BluePublicAllocator.sol#L50-L85](#)): no submit and accept, no delay.

Enabling `canAllocateFromIdle` can open a path that blocks all withdrawals. Depositors cannot react in time: an attacker can act right after the enable transaction and, in the same block, can sweep idle or the liquidity market so ordinary exits revert.

Impact

Depositors cannot observe the rise in risk and cannot exit before it takes effect.

Code Snippet

All the BluePublicAllocator grants are instant: [BluePublicAllocator.sol#L56-L62](#)

```
function setAbsoluteCap(address vault, address adapter, MarketParams calldata
↳ marketParams, uint256 newAbsoluteCap)
    external
{
    bytes32 id = vaultBlueId(adapter, marketParams);
    require(IVaultV2(vault).isAllocator(msg.sender), Unauthorized());
    absoluteCap[vault][id] = newAbsoluteCap;
```

Tool Used

Manual Review

Recommendation

Consider putting a delay on the BluePublicAllocator: mirror VaultV2's submit and accept pattern for the widening direction of every setter, keep the allocator role for narrowing only, e.g.:

```
mapping(address vault => mapping(bytes data => uint256)) public executableAt;
uint256 public constant WIDEN_DELAY = 1 days; // set per vault or fixed

function submitWiden(address vault, bytes calldata data) external {
    require(IVaultV2(vault).isAllocator(msg.sender), Unauthorized());
    require(executableAt[vault][data] == 0, AlreadyPending());
    executableAt[vault][data] = block.timestamp + WIDEN_DELAY;
    emit SubmitWiden(msg.sender, vault, data, executableAt[vault][data]);
}

function setAbsoluteCap(address vault, address adapter, MarketParams calldata
↳ marketParams, uint256 newAbsoluteCap)
    external
{
    bytes32 id = vaultBlueId(adapter, marketParams);
    if (newAbsoluteCap > absoluteCap[vault][id]) {
```

```

        // widen: must have been submitted and waited
        require(executableAt[vault][msg.data] != 0 && block.timestamp >=
            ↪ executableAt[vault][msg.data], Timelocked());
        executableAt[vault][msg.data] = 0;
    } else {
        // narrow: instant, allocator only
        require(IVaultV2(vault).isAllocator(msg.sender), Unauthorized());
    }
    absoluteCap[vault][id] = newAbsoluteCap;
    emit SetAbsoluteCap(msg.sender, vault, adapter, marketParams, newAbsoluteCap);
}

```

Alternative is documenting: state that granting the allocator role to this contract is itself the irreversible risk decision.

Discussion

MathisGD

In our model this is fine:

- if you don't trust allocators, then the BPA doesn't introduce anything new (it can act in the vault's caps and that's it)
- if you trust allocators, then you shouldn't use the BPA at all

we ack this issue

Issue M-4: BluePublicAllocator can be used to grief Vault liquidity [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit/2026-07-morpho-vault-v2-update-july-30th/issues/14>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

BPA's `canDeallocate` is per market, while `canAllocateFromIdle` is one flag for the whole Vault. An attacker can move all idle into a market with `canDeallocate == false` for the flat fee, i.e. there is no "the public may place funds only where it can also remove them" guarantee, and the cases when it's false can be used for griefing: reversal needs an allocator, until then every depositor exiting pays up to 2% through Vault's `forceDeallocate()`.

Vulnerability Detail

`allocateFromIdle()` checks only the active adapter, the Vault flag and the destination cap ([BluePublicAllocator.sol#L132-L138](#)). `canDeallocate` gates the source of `reallocate()` on a market level with `require(canDeallocate[vault][deallocateId], CannotDeallocate())`. So market can be an enabled destination without being a permitted source not by flag tuning operational mistake, but from lack of flag granularity.

Where exits depend on idle (no liquidity adapter, or a thin liquidity market), sweeping idle blocks the ordinary path ([VaultV2.sol#L809-L812](#)). The only exit left is `forceDeallocate()`, which fee is up to 2% ([VaultV2.sol#L839](#)), and stays in the Vault, so an attacker holding shares is paid by the depositors forced out, expanding this beyond pure griefing.

When the destination has `canDeallocate == false` no public call reverses the placement. The setup is asymmetric: a flat fee moves any amount into a place whose exit costs a percentage of assets, with no option but allocator intervention.

Impact

Depositors who need liquidity pay up to 2% per exit, an attacker holding shares collects a share of it, and this state persists until an allocator intervenes. Cost of attack is flat `nativePenalty` plus gas.

Code Snippet

The Vault wide `canAllocateFromIdle` flag is the only destination gate:

[BluePublicAllocator.sol#L132-L138](#)

```
require(isActiveAdapter[vault][adapter], InactiveAdapter());
require(vaultData[vault].canAllocateFromIdle, CannotDeallocate());

IVaultV2(vault).allocate(adapter, abi.encode(marketParams), assets);

bytes32 allocateId = vaultBlueId(adapter, marketParams);
require(IVaultV2(vault).allocation(allocateId) <= absoluteCap[vault][allocateId],
    ↪ AbsoluteCapExceeded());
```

Tool Used

Manual Review

Recommendation

Make allocate from idle per market so destination and source permissions can match, i.e. `canDeallocate[id] == false` whenever `canAllocateFromIdle[id] == false`, e.g.:

[BluePublicAllocator.sol#L133](#)

```
-   require(vaultData[vault].canAllocateFromIdle, CannotDeallocate());
+   bytes32 allocateId = vaultBlueId(adapter, marketParams);
+   require(canAllocateFromIdle[vault][allocateId], CannotAllocateFromIdle());
```

Discussion

MathisGD

we documented it [here](#). you can mark it as acknowledged

Issue L-1: Vault's forceDeallocatePenalty can be bypassed or decreased with BluePublicAllocator's reallocate() [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit/2026-07-morpho-vault-v2-update-july-30th/issues/4>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

Vault's `forceDeallocate()` charges up to 2% of the withdrawn amount, the only compensation remaining depositors get when someone exits an illiquid Vault. BluePublicAllocator's `reallocate()` performs the same deallocation with a flat fee paid to allocators instead of depositors, and can place the proceeds where Vault's `exit()` withdraws from, i.e. an exiting depositor can place their own exit liquidity for a flat fee, then withdraw free.

Also, a depositor can reallocate between adapters to lower the effective force deallocation penalty at the Vault's level. I.e. when one adapter has a lower `forceDeallocatePenalty[adapter]` than the other, a depositor can move the liquidity via BluePublicAllocator there first, then call `forceDeallocate()` for this market, gaming Vault's force deallocation to a degree.

Vulnerability Detail

`forceDeallocate()` deallocates then burns `assets.mulDivUp(forceDeallocatePenalty[adapter], WAD)` in shares from `onBehalf`, capped at 2% ([VaultV2.sol#L834-L839](#)); the penalty stays in the Vault for the remaining depositors.

`reallocate()` reaches the same `deallocateInternal()` ([BluePublicAllocator.sol#L114](#)) then allocates anywhere permitted. It doesn't require `forceDeallocatePenalty`, the whole price is `nativePenalty` ([BluePublicAllocator.sol#L106](#)) accrued to the allocator pot.

With the destination set to the market named by `liquidityData`, the assets land where

Vault's `exit()` looks ([VaultV2.sol#L809-L812](#)), with both getters being public. The exiting depositor can then withdraw avoiding burning penalty shares. This requires active adapters, `canDeallocate` on the source and unfilled caps.

Impact

Remaining depositors lose the compensation the mechanism exists to provide, while the exiting depositor and the allocator pot capture it (e.g. 20k on a 1M exit at 2% fee). It's repeatable and attack's economics improves with the amount.

For the adapter gaming vector it's a direct avoidance of the `forceDeallocatePenalty` configured at the Vault level for the adapter, i.e. the depositors lose the same compensation partially.

Code Snippet

BluePublicAllocator can provide the same deallocation functionality as Vault's `forceDeallocate()`, with no share penalty: [BluePublicAllocator.sol#L106-L118](#)

```
require(msg.value == vaultData[vault].nativePenalty, IncorrectNativePenalty());
require(canDeallocate[vault][deallocateId], CannotDeallocate());

IVaultV2(vault).deallocate(deallocateAdapter, abi.encode(deallocateMarketParams),
    ↪ assets);
IVaultV2(vault).allocate(allocateAdapter, abi.encode(allocateMarketParams), assets);
```

Tool Used

Manual Review

Recommendation

As a strict functionality partition consider forbidding the Vault's current liquidity market as a public destination, e.g.:

[BluePublicAllocator.sol#L115](#)

```

+   require(
+       allocateAdapter != IVaultV2(vault).liquidityAdapter()
+       || keccak256(abi.encode(allocateMarketParams)) !=
↪   keccak256(IVaultV2(vault).liquidityData()),
+       CannotAllocateToLiquidityMarket()
+   );
    IVaultV2(vault).allocate(allocateAdapter, abi.encode(allocateMarketParams),
↪   assets);

```

This closes the free exit path, but doesn't close the adapter gaming route, where the holder reallocates into a market of the adapter with the lowest `forceDeallocatePenalty` and then `forceDeallocate()` from there. To control for it consider evening out the penalty percentage levels across Vault and BluePublicAllocator, given that a proportional penalty described in the previous issue was implemented.

Discussion

MathisGD

we don't think that this needs to be fixed. we added a comment to remind that the vault's penalty is completely ignored:

<https://github.com/morpho-org/vault-v2/pull/965>

peyha

We ended up implementing a relative fee in loan token which can be set to fix this

<https://github.com/morpho-org/vault-v2/pull/966>

dmitriia

It looks good, although the arbitrage between BPA and Vault public deallocation paths needs to be monitored and controlled as BPA's penalty is still defined at Vault level, `vaultData[vault].penalty`, while Vault's penalty is per adapter, `forceDeallocatePenalty[adapter]`

Issue L-2: BluePublicAllocator can block deposits if sum of its per market caps exceeds a shared Vault cap [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit/2026-07-morpho-vault-v2-update-july-30th/issues/6>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

The Vault caps three ids per market: the whole adapter, the collateral token, and the market. The two shared ids are aggregates of the market ones: the same delta is applied to all three on every allocation ([VaultV2.sol#L588-L598](#)). The BluePublicAllocator (BPA) caps each market individually, so the public can fill a shared id up to the sum of the corresponding market caps. If that sum exceeds the Vault's shared cap, anyone can drive the shared id exactly to the Vault cap, after which every deposit reverts.

Vulnerability Detail

Each public allocation ends with `allocation(id2) <= absoluteCap[vault][id2]` ([BluePublicAllocator.sol#L117-L118](#)), so the public can push each market's recorded allocation to at most its BPA cap. The level of a shared id is the sum of the levels of its markets, so the public can push it to at most that sum. If the sum exceeds the Vault's shared cap (e.g. for this collateral), a normal configuration when market caps are sized independently (each market at X, collateral total at 2X), the attacker allocates until the shared id reaches the Vault cap, which passes at equality.

After that each deposit routes through `enter()` and `allocateInternal(liquidityAdapter, liquidityData, assets)` ([VaultV2.sol#L785](#)), adds to the saturated id and reverts with `AbsoluteCapExceeded`. Recovery needs an allocator to deallocate and/or update caps manually.

Impact

In a configuration where the sums are wrong, deposits and mints can be blocked across the Vault until an allocator intervenes, repeatably until caps are updated. Caps themselves hold, no deposited funds are stolen, this is a enter denying grieving vector.

Code Snippet

BPA per market checks sum up to the aggregates: [BluePublicAllocator.sol#L117-L118](#)

```
bytes32 allocateId = vaultBlueId(allocateAdapter, allocateMarketParams);
require(IVaultV2(vault).allocation(allocateId) <= absoluteCap[vault][allocateId],
↳ AbsoluteCapExceeded());
```

In Vault three ids are charged with the same delta, letting BPA overflow an aggregate on loose enough absoluteCap set: [MorphoMarketV1AdapterV2.sol#L267-L273](#)

```
function ids(MarketParams memory marketParams) public view returns (bytes32[]
↳ memory) {
    bytes32[] memory ids_ = new bytes32[](3);
    ids_[0] = adapterId;
    ids_[1] = keccak256(abi.encode("collateralToken",
↳ marketParams.collateralToken));
    ids_[2] = keccak256(abi.encode("this/marketParams", address(this),
↳ marketParams));
    return ids_;
}
```

Tool Used

Manual Review

Recommendation

Consider documenting the sizing rule: per collateral and per adapter, the sum of BPA market caps must not exceed the Vault's corresponding shared cap minus feasible headroom for the deposits. Optionally consider exposing the corresponding aggregate

via BPA logic so the allocators can check it when setting caps. Contract side alternative is to let the allocators cap the shared ids directly.

Discussion

MathisGD

we ack this issue. we added a line of documentation about it here

-><https://github.com/morpho-org/vault-v2/pull/967/changes>

allowbreak

#diff-0804a43f5b1314940fd474dc6ba6c7df270dc07074dcf4536dcf47d750b5c0aa

Issue L-3: One-wei allocations bypass zero absoluteCap due to rounding-down of reported assets [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-07-morpho-vault-v2-update-july-30th/issues/9>

Vulnerability Detail

Assume that `absoluteCap` is set to zero for a specific market to disable the public from pushing liquidity into it. However, the existing control does not entirely block liquidity from pushing into it.

For an adapter that has no existing position in the destination Morpho market, a one-unit supply can mint nonzero Morpho shares while the adapter's reported asset value remains zero.

Someone can call `allocateFromIdle` or `reallocate` with one-unit. Due to rounding, the reported asset is zero, and thus the allocation increases by zero. As a result, one raw asset unit can be moved from vault idle, or from the reallocation source market, into a destination market whose BPA absolute cap is zero.

```
require(IVaultV2(vault).allocation(allocateId) <= absoluteCap[vault][allocateId],  
    ↪ AbsoluteCapExceeded());
```

It's not profitable for a malicious user to do this, and the precondition is also restrictive (e.g., when there is no existing position in that particular Morpho market, and rounding behavior occurs only in certain valid market share-ratio states). So, this is more of a broken invariant (zero `absoluteCap` should block liquidity).

Impact

Permissionless callers can push dust from idle or source markets into a zero-cap market, breaking the invariant that zero `absoluteCap` fully blocks public inflows.

Code Snippet

<https://github.com/sherlock-audit/2026-07-morpho-vault-v2-update-july-30th/blob/90a548423f83ae90942d0dbc49e3b53bb58ecf6a/BluePublicAllocator.sol#L110>

Tool Used

Manual Review

Recommendation

Consider adding `require(absoluteCap[vault][allocateId] > 0)` to short-circuit the zero-cap instead of proceeding with the deallocation and allocation steps.

Discussion

MathisGD

vault-v2 has the same issue (notably with the liquidity adapter, but also doable by any allocator), and this is fixed by [this](#). it fixes also the case you are mentioning. so the issue is invalid?

xiaoming9090

vault-v2 has the same issue (notably with the liquidity adapter, but also doable by any allocator), and this is fixed by [this](#). it fixes also the case you are mentioning. so the issue is invalid?

@MathisGD This issue is related to the BPA's cap (`absoluteCap[vault][allocateId]`), while the one [here](#) is the VaultV2's cap (`caps[id].absoluteCap`), and they independent of each other. The same "type" of issue exists in VaultV2 and it is mitigated by [this](#) guard, but that guard doesn't really mitigate this issue.

This issue is about the scenario where vault cap > 0, BPA cap = 0 (a market that allocators don't want the public pushing into). If BPA cap is zero for a specific, nothing should ever get pushed to that specific market. However, in the current implementation, dust can still be pushed into the market through the public allocator despite its zero BPA cap.

peyha

Good catch, having the invariant is nice so we ended up adding a cap check in `reallocate` and `allocateFromIdle` in [this PR](#)

Issue L-4: CALLVALUE/msg.value returns zero on Tempo [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-07-morpho-vault-v2-update-july-30th/issues/10>

Vulnerability Detail

Morpho protocol is deployed on Tempo. However, the issue is that Tempo is an EVM chain with no native token. Its EVM spec states that CALLVALUE, the opcode sitting underneath Solidity's msg.value, always returns zero.

Refer to <https://tempo.xyz/developers/docs/quickstart/evm-compatibility#balance-operations-and-rpc-methods> for more information.

If BluePublicAllocator is deployed on Tempo, it will not work as msg.value always returns zero, and the require penalty fee check will always fail and revert.

```
File: BluePublicAllocator.sol
098:     function reallocate(
099:         address vault,
100:         address deallocateAdapter,
101:         MarketParams calldata deallocateMarketParams,
102:         address allocateAdapter,
103:         MarketParams calldata allocateMarketParams,
104:         uint128 assets
105:     ) external payable {
106:         require(msg.value == vaultData[vault].nativePenalty,
↪ IncorrectNativePenalty());
```

```
File: BluePublicAllocator.sol
125:     function allocateFromIdle(address vault, address adapter, MarketParams
↪ calldata marketParams, uint128 assets)
126:         external
127:         payable
128:     {
```

```
129:         require(msg.value == vaultData[vault].nativePenalty,  
↪ IncorrectNativePenalty());
```

Impact

Public reallocation is permanently DoSed on Tempo, and the penalty-fee mechanism cannot function.

Code Snippet

<https://github.com/sherlock-audit/2026-07-morpho-vault-v2-update-july-30th/blob/90a548423f83ae90942d0dbc49e3b53bb58ecf6a/BluePublicAllocator.sol#L106>

<https://github.com/sherlock-audit/2026-07-morpho-vault-v2-update-july-30th/blob/90a548423f83ae90942d0dbc49e3b53bb58ecf6a/BluePublicAllocator.sol#L129>

Tool Used

Manual Review

Recommendation

On chain that does not support native-token transfers (`msg.value`), consider making the penalty payable in a configurable ERC-20 token (pulled via `transferFrom`). Otherwise, document that should be kept to `nativePenalty = 0` on such chain.

Discussion

MathisGD

indirectly fixed by <https://github.com/morpho-org/vault-v2/pull/966>

Issue L-5: Permissionless deallocation allows repeated draining of withdrawal liquidity blocking user exits [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit/2026-07-morpho-vault-v2-update-july-30th/issues/11>

This issue has been acknowledged by the team but won't be fixed at this time.

Vulnerability Detail

Assume that the vault has two markets (Market A and B). The vault's liquidityAdapter and liquidityData are set to deposit and withdraw to Market A automatically when the user deposits or withdraws.

Market A has 1,000,000 USDC allocation while Market B has none at the moment. Market A is set in `canDeallocate`, so anyone can deallocate from Market A and reallocate to an approved market.

Assume that Alice (innocent user) tries to withdraw 100,000 USDC. A malicious user could front-run and reallocate 1,000,000 USDC from Market A to Market B. Thus, when Alice's TX gets executed, it will revert because there are 0 supply shares on Market A.

Allocator can fix this by reallocating 1,000,000 USDC from Market B to Market A, but a malicious user can repeat the same again.

Impact

Anyone can repeatedly deallocate all funds out of the market backing the vault's liquidity adapter, causing pending user withdrawals to revert.

The allocator have `nativePenalty` mechanism which reduce the risk of this issue. However, the penalty is a flat fee, not proportional to the amount moved, so the cost of the attack does not scale with the harm.

Code Snippet

<https://github.com/sherlock-audit/2026-07-morpho-vault-v2-update-july-30th/blob/90a548423f83ae90942d0dbc49e3b53bb58ecf6a/BluePublicAllocator.sol#L98>

Tool Used

Manual Review

Recommendation

Consider having per-market `minAllocation` floor or flow-cap (e.g., `maxIn/maxOut` per market) which might be helpful but it requires additional implementation. Otherwise, consider documenting this for awareness.

Discussion

MathisGD

we acknowledge this issue. allocators should set the BPA's config accordingly ->
<https://github.com/morpho-org/vault-v2/pull/967>

Issue L-6: BluePublicAllocator can list or allocate to any enabled but unused market [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit/2026-07-morpho-vault-v2-update-july-30th/issues/12>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

MorphoMarketV1AdapterV2 appends a market to `marketIds` when its allocation goes from zero to nonzero, and `realAssets()` walks that array on the first Vault touch of every transaction. Any public allocation into a market with caps above zero triggers the append. The caller pays only the fixed `nativePenalty` and gas. This means that anyone can place any enabled market into the active set, inflating the cost of every Vault deposit, mint, withdraw and redeem.

Vulnerability Detail

The append fires on the transition from zero to nonzero ([MorphoMarketV1AdapterV2.sol#L242-L244](#)). `accrueInterestView()` calls `realAssets()` for every adapter ([VaultV2.sol#L667-L670](#)) on deposit, mint, withdraw, redeem, allocate, `forceDeallocate()` and `setMaxRate()`.

Preconditions: BluePublicAllocator's cap above zero ([BluePublicAllocator.sol#L138](#)) and Vault absolute cap above zero on every id the adapter returns, else `allocateInternal()` reverts ([VaultV2.sol#L592](#)).

Entries can be sticky: the adapter drops a market from `marketIds` only when its computed value hits exactly zero. That needs a deallocation that burns the position with no rounding residue and enough free liquidity on Morpho; heavily used markets can make the withdrawal revert. I.e. removal is manual (allocator only), can require some supportive operations, and the attacker can re-list for the fixed fee while initial config stays.

A market on an enabled adapter may be compromised in a way manual allocators catch; the public channel can still allocate into it. Restricting entry to markets the Vault is already in keeps the market set an allocator decision.

The adapter documents that a reverting `expectedSupplyAssets()` freezes accrual. That was safe only while entry required the allocator role. A freeze is not reachable through this vector alone: the valuation never reads the oracle and the IRM call is skipped when `totalBorrowAssets == 0` ([AdaptiveCurveIrmLib.sol#L38](#)).

If the penalty later becomes proportional to `assets`, the listing still works with a dust move: the transition is amount independent, so even then the attacker pays almost nothing and the permanent gas surcharge remains.

Impact

Every depositor pays a permanent gas surcharge proportional to the number of markets listed, and enough entries price out small deposits and withdrawals. Attacker spending is gas plus `nativePenalty`. Recovery is market by market through allocator deallocations that almost fully used markets can revert.

Code Snippet

Append happens on the first nonzero allocation:

[MorphoMarketV1AdapterV2.sol#L233-L244](#)

```
function updateList(bytes32 marketId, uint256 oldAllocation, uint256 newAllocation)
↪ internal {
    if (oldAllocation > 0 && newAllocation == 0) {
        // ... remove from marketIds
    } else if (oldAllocation == 0 && newAllocation > 0) {
        marketIds.push(marketId);
    }
}
```

The Vault rejects a zero absolute cap: [VaultV2.sol#L592](#)

```
require(_caps.absoluteCap > 0, ErrorsLib.ZeroAbsoluteCap());
```

Tool Used

Manual Review

Recommendation

Consider refusing public entry into a market the Vault is not already in, e.g.:

[BluePublicAllocator.sol#L135](#)

```
+   require(IVaultV2(vault).allocation(vaultBlueId(adapter, marketParams)) > 0,  
↪   MarketNotEntered());  
       IVaultV2(vault).allocate(adapter, abi.encode(marketParams), assets);
```

Discussion

MathisGD

we acknowledge this issue. allocators can configure the PA in such a way that this doesn't happen, as anyway an allocator could already do this, and we don't think that the behavior is misleading.

Issue L-7: Curator and sentinel cannot narrow or stop the public allocator [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit/2026-07-morpho-vault-v2-update-july-30th/issues/13>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

All six setters of BluePublicAllocator control access via `IVaultV2(vault).isAllocator(msg.sender)`. Curator, sentinel and owner have no access, so the settings that scope the public allocations are writable only by the same role they are to contain.

Vulnerability Detail

Every setter carries the same access control ([BluePublicAllocator.sol#L51](#)). VaultV2 does the opposite for risk reduction: `decreaseAbsoluteCap()` accepts curator or sentinel at once ([VaultV2.sol#L535-L543](#)); `deallocate()` accepts allocator or sentinel ([VaultV2.sol#L602-L605](#)).

A sentinel can `deallocate()` to idle, but `allocate()` is open to allocators only ([VaultV2.sol#L575-L578](#)), so the public risks the market again at once. Zeroing the cap of the whole adapter blocks the public (the cap must be above zero, [VaultV2.sol#L592](#)) but also halts deposits through the same check ([VaultV2.sol#L785](#)), so the only fast stop is a deposit halt across the Vault, undone only through timelocked `increaseAbsoluteCap()`. Removing the contract runs through timelocked `setIsAllocator()` ([VaultV2.sol#L381-L385](#)).

Impact

In an incident the emergency responder can either do nothing about the public allocator or halt all the deposits across the Vault, with timelocked recovery and with revocation locks the delegation open.

Code Snippet

BPA has uniform control on every setter: [BluePublicAllocator.sol#L50-L54](#)

```
function setIsActiveAdapter(address vault, address adapter, bool
↪ newIsActiveAdapter) external {
    require(IVaultV2(vault).isAllocator(msg.sender), Unauthorized());
    isActiveAdapter[vault][adapter] = newIsActiveAdapter;
    emit SetIsActiveAdapter(msg.sender, vault, adapter, newIsActiveAdapter);
}
```

Tool Used

Manual Review

Recommendation

Consider accepting curator and sentinel for the restricting direction of every setter, keeping the allocator role for widening, e.g.:

[BluePublicAllocator.sol#L51](#)

```
-    require(IVaultV2(vault).isAllocator(msg.sender), Unauthorized());
+    require(
+        IVaultV2(vault).isAllocator(msg.sender)
+        || (!newIsActiveAdapter
+            && (msg.sender == IVaultV2(vault).curator() ||
↪ IVaultV2(vault).isSentinel(msg.sender))),
+        Unauthorized()
+    );
```

Discussion

MathisGD

we acknowledge this issue. note that curators can't allocate/deallocate in the vault as well (but they can add themselves as a allocator). also sentinels/curators can set the

caps to zero in case of emergency (which effectively deactivates the PA)

Disclaimers

Blackthorn does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.